

Construction Element Detection for Site Progress Verification

A cloud-only YOLOv8 pipeline for automated progress evidence

Repository: github.com/yazandarwish264-glitch/m4u3-construction-detection

Progress evidence is captured, then lost

- **Latency.** Two to seven days between the photograph and the progress record. Decisions get made on stale data.
- **Inconsistency.** Two engineers describe the same floor differently. No controlled vocabulary links what is visible to a structured record.
- **Loss of information.** The photograph is archived. What is *in* it — steelbar placed, blockwork started, conduit run — is never captured as data, so it cannot enter a CDE or be audited.

In the Gulf, progress reporting on large programmes is still photograph-plus-narrative, with a two-to-seven-day lag between capture and record. Government BIM mandates and ISO 19650 information-delivery requirements are raising the bar on structured, auditable progress evidence. A detection record — image, time, zone, class, confidence — is structured information that can enter a CDE. A JPEG in a folder is not.

Five classes, one detector, zero local installs

Detect construction elements and plant in site photographs, then use presence as a progress and site-condition signal.

Class	Signal it carries
steelbar	Reinforcement activity — structural progress
brick	Masonry and blockwork — envelope progress
pvcpipe	Services installation — MEP first fix
scaffold	Temporary works standing — zone not released
excavator	Plant present — earthworks active

Pipeline

- Roboflow — fork, extend, annotate
- Colab — YOLOv8, 30 epochs, T4
- Validate — P / R / mAP, per class
- Evidence — validation + unseen images
- Handover — repo, governance, licence

A public base, re-split for the task

Forked [seungyeon/construction-site-km7bh](#) (700 images, CC BY 4.0) into our own Roboflow workspace and re-split it to the required 80/20. Used as published, with no images added. Seven photographs of our own are held back entirely.

Dataset	As used in this project
Source	seungyeon/construction-site-km7bh — CC BY 4.0, redistributed with attribution
Images	700, all annotated, five classes
Split	560 train / 140 validation (80 / 20), re-split from the source's 70/20/10
Preprocessing	Resize to 640 x 640. No augmentation
Held out entirely	7 first-party photographs, never uploaded and never trained on

Two decisions that shape everything after this slide. Augmentation is off, so the baseline shows what the data does rather than what augmentation hides. And the split is random over images — but this dataset is built from sequential site photographs, so near-identical frames land on both sides of it. That is why the score on the next slide is not a generalisation estimate, and why the seven held-out photographs exist.

Validation performance

0.950

Precision

0.884

Recall

0.943

mAP@50

0.809

mAP@50-95

Class	P	R	mAP@50	mAP@50-95
brick	0.984	1.000	0.995	0.932
excavator	0.969	1.000	0.995	0.883
pvcpipe	0.986	0.930	0.964	0.835
scaffold	0.955	0.875	0.979	0.858
steelbar	0.855	0.612	0.781	0.535

Success criteria, all met. S1 overall mAP@50 ≥ 0.50 — **0.943**. S2 steelbar recall ≥ 0.50 — **0.612**. S3 brick recall ≥ 0.40 — **1.000**. Targets were set before the run. Clearing them by this margin is a signal that the measurement is wrong, not that the model is excellent — see the next slide.

Three things the numbers say

- 1 Validation says 0.943 mAP@50. Our own photographs say almost nothing.** Across seven held-out images the model produced one marginal correct detection, two confident wrong-class detections and four blanks — including both classes we had designated as controls at recall 1.000.
- 2 The cause is scale, and it is testable.** Holding weights, image and confidence fixed and changing only the input resolution, a blockwork wall moves from scaffold 0.78 to brick 0.46. A model whose class prediction depends on resampling has learned a texture signature, not an object. Lowering confidence to 0.02 recovers nothing.
- 3 The fix is data, not capacity.** Re-split by scene group so validation can detect this at all; add wide-field imagery and train multi-scale; add hard negatives for the two confusions actually observed. A larger model would raise the leaking number and change none of the seven held-out results — a third run, bigger architecture, different split, put steelbar last again.

The two errors are not symmetrical

False negative

Severity: High

Steelbar is placed, the model misses it. No record is created. Nobody investigates, because nobody knows there is anything to investigate. **The error is silent.**

False positive

Severity: Moderate

The model reports steelbar that is not there. The next site walk contradicts it. **The error is loud and self-correcting** — but repeated, it produces alert fatigue.

One model, two thresholds

Progress logging runs at a **low** confidence threshold, favouring recall because a miss is silent. Site-condition alerts run at a **high** one, favouring precision because false alarms train people to ignore the system. A single threshold would be wrong for one of them.

Three false positives and three false negatives are documented with filenames and hypotheses in *docs/error_analysis.md*, each tied to a dataset gap rather than to model capacity — and each paired with a concrete improvement and the metric it should move.

When not to use this

- Reported metrics are inflated by train/validation leakage; the held-out set confirms it independently. They measure memorisation, not generalisation.
- Predictions change class with input resolution, so imgsz must be pinned end to end and confidence is not comparable across distributions.
- Wide-field shots fail — which is how site progress photography is actually taken. Validated only on close, subject-filling images.
- No safety function: no people, no PPE, no hazards. Treating it as a safety system would reduce site safety, not improve it.

Rights

CODE
MIT

DATASET
CC BY 4.0, not owned

WEIGHTS
AGPL-3.0 inherited

PEOPLE DETECTED
None, by design

Commercial deployment of YOLOv8-derived weights requires AGPL-3.0 compliance or an Ultralytics licence. This is the most commonly missed obligation in YOLO projects.